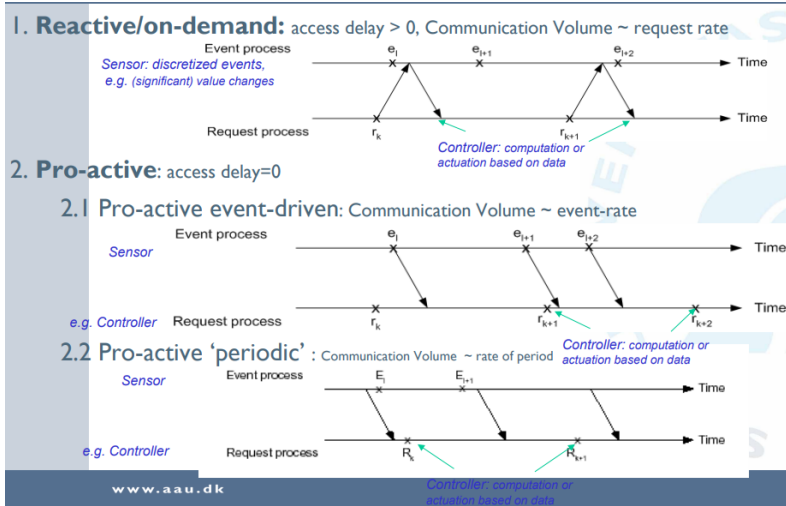


When a control process needs information from a sensor (event process $\mathcal{E}$), we must design or choose a **data access scheme** — deciding how and when data flows to the controller. We look at **3 strategies**:

- **Reactive:** controller sends a request $\mathcal{R}$ to the sensor when it needs data — pull on demand (not periodic, triggered by need, modelled as Poisson rate μ).
- **Proactive event-driven:** sensor pushes an update to the controller every time an event occurs (value changes) — no request needed.
- **Proactive periodic:** sensor pushes on a fixed schedule $\mathcal{P}$ regardless of whether the value changed.

1 Access Strategies and Processes

Each strategy can be described and modelled using the processes of :



Processes:

- $\mathcal{E}$ (rate λ): event process at sensor
- $\mathcal{R}$ (rate μ): request process (reactive only)
- $\mathcal{D}$ (rate ν): downstream delay sensor $\rightarrow$ ctrl
- $\mathcal{U}$: upstream delay (reactive only)
- $\mathcal{P}$ (rate τ): period (proactive periodic only)

Together they capture how data moves from sensor to controller.

Assumption:

- All processes are mutually independent
- Message re-ordering can occur in network.

To compare strategies we need a measure of **data quality** — how well does the controller's knowledge match the true sensor value at the moment of computation? For this we use **mmPr**:

Which is the Probability that controller value does not match sensor value, described by:

$$\text{mmPr} := \Pr(X_C(C_i) \neq X_s(C_i))$$

- $X_s(C_i)$: true sensor value at computation instant C_i
- $X_C(C_i)$: controller's knowledge of sensor value at C_i

Goal: minimise mmPr $\approx$ minimise accumulated turbine damage.

2 Analysis: mmPr Formulas

We analyse mmPr for the 3 access schemes under the assumptions:

- Poisson event process at sensor (rate λ)
- General downstream delay $\mathcal{D}$ with distribution f_D (rate ν for exponential case)

Result: two of the three schemes yield the **same** mmPr — and the periodic scheme introduces a fundamental floor independent of network speed.

Reactive & Proactive event-driven — surprisingly same result.

Since all processes are independent, $\mathcal{R}$ is **irrelevant** for stationary mmPr — both ask the same question:

Reactive: did a new event occur while waiting for the response?

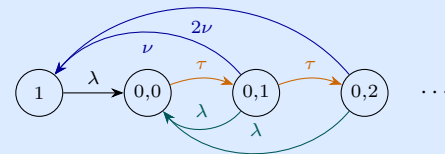
Proactive (Full updates): did a new event occur while the last update was in transit?

Both reduce to: *did the most recent message arrive before the next event?*

$$\text{mmPr} = 1 - \mathcal{L}\{f_D\}(\lambda) \xrightarrow{\text{exp. delay}} \frac{\lambda}{\lambda + \nu}$$

Large ν : mmPr $\rightarrow$ 0. Large λ : mmPr $\rightarrow$ 1.

Proactive periodic — sensor sends at fixed rate τ . More complex than reactive since packets are sent continuously $\Rightarrow$ **multiple in transit** simultaneously. Model with a **CTMC** tracking packets in flight.



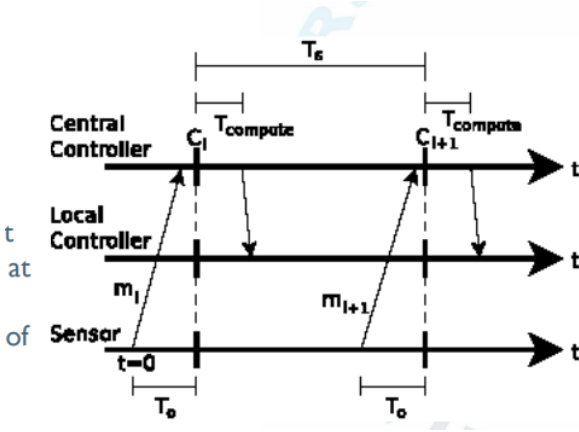
- **1**: matching;
- **0,0**: MM, 0 pkts sent, no in transit but event has happened;
- **0,k**: MM, k in transit.

Lowest mmPr $\rightarrow \lambda/(\lambda + \tau)$ as $\nu \rightarrow \infty$. τ **always dictates lower bound** — even if network is infinitely fast.

Note — incremental (not used in presentation): proactive event-driven *incremental updates* = MM $\Leftrightarrow \mathcal{E}/\mathcal{D}/\infty$ queue busy $\Rightarrow$ mmPr = $1 - e^{-\lambda/\nu}$ (Poisson). Delay-distribution-independent.

3 Wind-Farm Timing Model: Offset T_o

Even though proactive periodic has the worst mmPr floor ($\lambda/(\lambda+\tau)$) on fast networks, it is used in the wind-farm because the system demands deterministic, scheduled communication. This means we must choose not only the period $T_s = 1/\tau$ but also **when** within the period to send — controlled via offset T_o .



$T_s = 150$ ms, $T_{\text{compute}} = 50$ ms, $T_o = \text{offset}$ (sensor sends T_o before C_i).

Trade-off:

- larger $T_o \Rightarrow$ more time for delivery (fewer misses) but larger info age (data may have changed).
- small $T_o \Rightarrow$ less time for delivery (higher miss risk) but fresher data (low info age).
- Optimal $T_o \approx 100$ ms balances both.

Simulation can find optimal T_o but is **very expensive**: must run a full control simulation for every candidate T_o value.

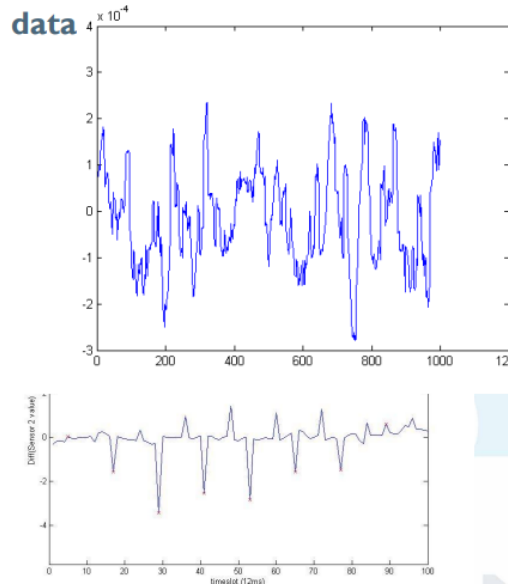
Instead: **Markov chain data quality model** — compute $\text{mmPr}(\varepsilon)$ analytically as a function of T_o , then minimise directly. Much faster; gives same optimal T_o .

Here $\text{mmPr}(\varepsilon)$ is the full threshold version:

$$\text{mmPr}(\varepsilon) := \Pr(|X_s(C_i) - X_C(C_i)| > \varepsilon)$$

ε is a percentile threshold on the sensor value error

4 Data Quality Model: Markov Chain Approaches



Why Markov? Sensor data has **two regimes**:

- (1) **Normal**: gradual drift between actuations.
- (2) **Actuation event**: sudden large jump (downward peaks in diff. series).

$\Rightarrow$ Discretise sensor value space into N states; model transitions as Markov chain.

Approach I (empirical):

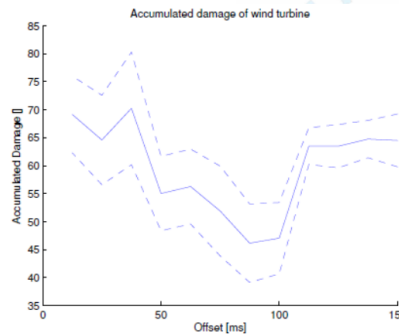
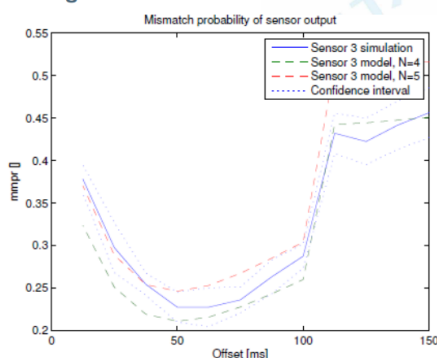
- Run control simulation (e.g. ideal scenario)
 - Take all sensor measurements $\Rightarrow$ snapshot
 - Discretise into N bins; estimate $N \times N$ transition matrix directly from data
 - Steady-state $\pi \Rightarrow$ compute $\text{mmPr}(\varepsilon)$
- Simple, but still requires a full simulation run.

Approach II (analytical):

- Same N -state discretisation
- Separate matrices: P (normal transitions) + C (actuation jumps)
- $1 - \text{mmPr}(\varepsilon) = F_D(T_o) \sum_k \pi_u(k) \cdot \Pr(k_i | k_{i-1})$
- Fully analytical as function of $T_o \Rightarrow$ minimise directly

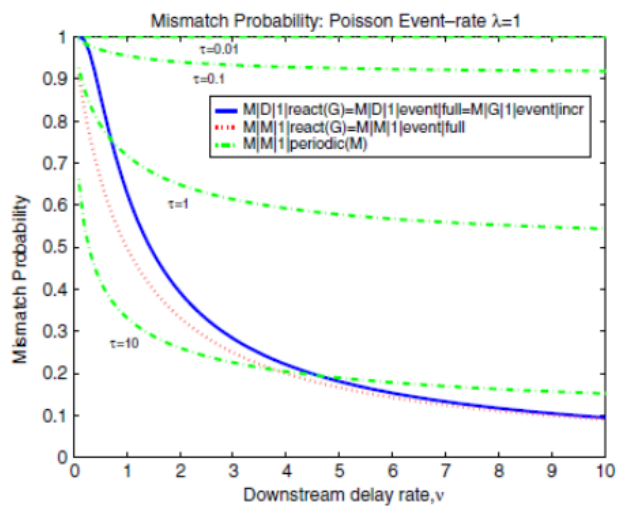
Heuristic: $T_o = p$ -percentile of delay ($p \approx 90\%$).

5 Controller Performance & Data Quality Validation



Conclusion: Markov model at $N=4$ or 5 already closely matches simulation — low N is sufficient.

mmPr $\neq$ min. damage exactly — damage curve has its minimum at similar T_o but not identical. Use mmPr as a fast analytical **indicator** to locate optimal T_o without expensive damage simulations.



Key results ($\lambda = 1$):

Det. delay D : reactive = event-driven full = incremental [blue]:

$$\text{mmPr} = 1 - e^{-\lambda D}$$

Incremental is delay-distribution-independent.

Exp. delays [red]: $\text{mmPr} = \lambda/(\lambda + \nu)$ — *better* than det.

Periodic [green, τ -labelled]: converges to $\lambda/(\lambda + \tau)$ for large ν — *worse* because $\tau \ll \nu$.